ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

ассистент		А. Э. Зянчурин
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №3

Интеграция общего (системного) и специального программного обеспечения в программных комплексах.

по курсу: Основы программной инженерии

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4434		К. А. Энгельгардт
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

СОДЕРЖАНИЕ

ЗАДАНИЕ 5

ВВЕДЕНИЕ..... 6

1 Лабораторная работа №3 — реализация и интеграция..... **Error! Bookmark not defined.**

1.1 Запуск 7

1.2 Структура и поведение..... **Error! Bookmark not defined.**

1.3 Локальная разработка без Docker.... **Error! Bookmark not defined.**

1.4 Демонстрация ..... **Error! Bookmark not defined.**

1.5 Проверки и эксплуатация..... **Error! Bookmark not defined.**

1.6 Роли и выводы ..... **Error! Bookmark not defined.**

2 Контракт API 9

3 Проектирование и эксплуатация базы данных**Error! Bookmark not defined.**

3.1 Состав таблиц и связи..... **Error! Bookmark not defined.**

3.2 Роли и доступ ..... **Error! Bookmark not defined.**

3.3 Индексы для порядка 100 тысяч объектов**Error! Bookmark not defined.**

3.3.1 Оценка ёмкости ..... **Error! Bookmark not defined.**

3.4 Транзакции и ограничения..... **Error! Bookmark not defined.**

3.5 Резервное копирование и восстановление**Error! Bookmark not defined.**

4 Проверка ЛР №3..... **Error! Bookmark not defined.**

Серверные интеграционные тесты..... **Error! Bookmark not defined.**

Проверки клиента..... **Error! Bookmark not defined.**

Нагрузочный набор и измерения..... **Error! Bookmark not defined.**

Системная интеграция ..... **Error! Bookmark not defined.**

Автоматический отдельный тестовый стенд**Error! Bookmark not defined.**

ЗАКЛЮЧЕНИЕ 12

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 13

ПРИЛОЖЕНИЕ А Результаты проверки интерфейса**Error! Bookmark not defined.**

ЗАДАНИЕ

Разработка серверной части информационной системы супермаркета на базе FastAPI: архитектура, контракт REST API, валидация входных данных, двухфазное проведение чека и интеграционное тестирование.

Задание выдал: _____

Подпись: _____

Задание принял: _____

Дата: _____

ВВЕДЕНИЕ

Работа реализует принятые требования супермаркета на React, FastAPI и PostgreSQL. Основные проверяемые свойства — серверная валидация, разграничение доступа, сохранение исторических цен, атомарная оплата и постраничная работа с большим каталогом.

Основание выполнения — соответствующая методичка [1]. Порядок оформления сверяется с нормативной документацией ГУАП [2].

1 Развертывание и структура серверной части

Участники: A — Кочетков Роман; B — Энгельгард Кирилл; C — Прохоров Максим.

Вариант 36: информационная система супермаркета. Здесь находятся прикладные исходники и схема БД. Требования и архитектура приняты на этапе ЛР №1, среда и порядок совместной работы подготовлены на этапе ЛР №2.

1.1 Запуск

В Linux VM с Docker Compose v2 и openssl:

```
cd 3lab
bash setup.sh
docker compose --env-file .env ps
curl --fail http://127.0.0.1:8083/api/health
```

Интерфейс: http://127.0.0.1:8083. OpenAPI/Swagger: http://127.0.0.1:8083/api/docs. Пользователи: manager, cashier, auditor. Их пароли сгенерированы в локальном .env в полях MANAGER_PASSWORD, CASHIER_PASSWORD, AUDITOR_PASSWORD. Просматривать их локально, не включать в Git и скриншоты отчёта. Они не зашиты в приложение. Токен хранится только в памяти страницы; обновление страницы требует повторного входа.

Начальная загрузка создаёт 16 демонстрационных товаров, трёх пользователей и движения начального остатка. Это небольшой набор для знакомства с UI; отдельный сценарий 100 тысяч объектов находится в db/seed_large.sql. Повтор bootstrap не меняет существующие пароли, карточки или остатки. Товарные цены и данные вымышленные учебные.

Используются Python 3.12, FastAPI, Pydantic 2, psycopg 3, PostgreSQL 17, React 18.3, JavaScript/JSX, PropTypes, Vite и Nginx. React 18.3 выбран для автоматических проверок PropTypes в development; в React 19 они удалены. В production дополнительная проверка JSON в api.js остаётся обязательной.

1.2 Локальная разработка серверной части

Создать отдельную PostgreSQL-БД supermarket, запустить init.sql администратором кластера. Пароли передавать переменными psql app_password, admin_password; файл не содержит готовых паролей. Для автоматизации предпочтителен включённый контейнерный bootstrap. Затем:

```
cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
export DB_HOST=127.0.0.1 DB_PORT=5432 DB_NAME=supermarket DB_USER=market_app
read -rs -p 'Пароль market_app: ' DB_PASSWORD; export DB_PASSWORD
```

1.3 Проверки и эксплуатация

Проверка работоспособности и эксплуатационных характеристик системы проводилась по многоуровневому регламенту, включающему серверные интеграционные тесты, проверку доступности клиентского интерфейса и нагрузочные измерения базы данных. Персистентность данных при перезапуске контейнеров обеспечивается внешним томом СУБД. При модернизации структуры данных регламент предусматривает предварительное создание резервной копии и последовательное применение миграций. Скрипт инициализации схемы ориентирован на первичное развертывание чистой базы данных.

Пакет является учебной версией. Для внешней эксплуатации нужны TLS, отдельный процесс управления сотрудниками, регламент мониторинга, резервного копирования и ротации секретов. Приложение слушает localhost по умолчанию. Swagger подгружает UI-ресурсы с CDN; сама схема /api/openapi.json доступна без CDN.

2 Контракт API

Базовый путь /api. Защищённые методы требуют Authorization: Bearer TOKEN. Токен выдаётся POST /auth/login, сохраняется на сервере в виде SHA-256 и действует 8 часов. POST /auth/logout отзывает текущую сессию; GET /auth/me возвращает идентификатор, имя и роль. Пароль проверяется scrypt с индивидуальной солью. Лимит попыток входа настроен в Nginx, API не опубликован напрямую.

Метод и путь	Роль	Результат
GET /health	Без входа	Проверка соединения с БД
POST /auth/login	Без входа	access_token, token_type, expires_at
GET /auth/me	Любая	Текущий сотрудник
POST /auth/logout	Любая	204, токен отозван
GET /products	Любая	items и next_cursor
POST /products	manager	201, созданная карточка с stock=0
PUT /products/{id}	manager	Полная карточка с новым version
POST /products/{id}/stock	manager	Новый остаток и версия
POST /receipts	cashier, manager	201, черновик или прежний результат ключа
GET /receipts	Любая	Страница истории; для кассира только собственные
GET /receipts/{id}	По правам чтения	Чек, строки, итог
POST /receipts/{id}/pay	cashier, manager	Оплаченный чек; повтор безопасен
POST /receipts/{id}/cancel	cashier, manager	Отменённый черновик

Каталог: q — подстрока названия от 3 символов либо полный цифровой штрихкод длиной 8–14; category — точное название категории; after — предыдущий ID, по умолчанию 0; limit — 1–100, по умолчанию 24; include_archived=true доступен менеджеру. Порядок всегда по ID ASC. Для истории — before и ID DESC. next_cursor=null означает конец. Пагинация не представляет снимок БД между запросами: параллельные вставки или изменения могут изменить видимый набор, но offset-смещения нет.

Пример создания карточки:

```
{"barcode":"4601234567890","name":"Молоко 1 л","category":"Молочные продукты","price":"99.90"}
```

При PUT передаются те же поля, active и текущий version. Начальный остаток 0; корректировка выполняется отдельно:

```
{"delta":10,"reason":"Учебное поступление"}
```

Создание чека (ID товара взять из реального GET/POST):

```
{"request_id":"78c9bf7f-06e5-451a-a81a-e528537296e8","items":[{"product_id":1,"quantity":2}]}
```

Цена, итог, статус, автор, ID чека и paid_at не принимаются от клиента. Неизвестные поля запрещены. Повтор с тем же request_id и тем же составом возвращает тот же документ, в том числе если он уже завершён. При неизвестном исходе запроса нельзя автоматически генерировать другой UUID.

Сервер возвращает цены/суммы строками Decimal ("199.80"), даты — ISO 8601 с часовым поясом. Корзина браузера имеет только предварительную сумму. Гарантия исторической цены начинается после создания серверного чека.

Ошибки валидации имеют detail: [{loc, msg, type}] без исходного тела и пароля. Бизнес-ошибки имеют detail: "сообщение". 409 не повторяют бесконечно: нужно обновить данные или отменить черновик. 503 при блокировке допускает повтор того же идентификатора. 401 означает необходимость повторного входа.

3 Серверное интеграционное тестирование

3.1 Проведение и верификация серверных тестов

Тесты используют настоящую PostgreSQL, а не SQLite. Нужна отдельная БД с именем, заканчивающимся _test, и схема init.sql. На сервере с несколькими тестовыми базами учитывать, что роли PostgreSQL относятся ко всему кластеру: создавать их один раз, а дальнейшие схемы восстанавливать из проверенной копии.

DATABASE_URL — строка подключения market_app, TEST_ADMIN_URL — market_admin в той же отдельной базе. Передать секреты через окружение. Из backend:

```
python -m unittest discover -s tests -v
```

Без переменных тесты явно пропускаются и не означают успешную интеграционную проверку. Fixture создаёт пользователей test_manager, test_cashier, test_auditor и меняет только их пароли в _test-БД. Товары имеют случайные штрихкоды; существующие таблицы не очищаются. После тестирования временный стенд можно удалить целиком отдельно от рабочих данных.

Проверяются: аутентификация/выход, роли, некорректные деньги и количества, чужие поля, пустой и дублированный состав, ограничение пагинации, SQL-подобная строка поиска, повтор UUID, повтор pay, снимок цены, конкуренция за последний товар, переходы состояний, архивирование, конфликт версии карточки и ограниченные SQL-права.

ЗАКЛЮЧЕНИЕ

Реализованы база данных, модульный FastAPI API и компонентный React-интерфейс супермаркета. Серверная валидация, роли и транзакции отделены от HTTP-маршрутов; вход, каталог, кассовая панель и история оформлены отдельными компонентами. Большой набор данных, планы SQL и восстановление копии проверены. Фактические результаты повторных тестов и границы контейнерной интеграции приведены в общем протоколе проверки.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Основы программной инженерии. Методические указания к выполнению лабораторной работы № 3 / сост. П. А. Охтилев, А. Э. Зянчурин. Санкт-Петербург : ГУАП, 2021. Электронный документ, предоставленный к заданию.

2. ГУАП. Для учебного процесса. Нормативная документация. URL: https://guap.ru/c/regdocs/docs/uch (дата обращения: 12.09.2026).

3. FastAPI. Documentation. URL: https://fastapi.tiangolo.com/ (дата обращения: 12.09.2026).

4. PostgreSQL 17. Documentation. URL: https://www.postgresql.org/docs/17/ (дата обращения: 12.09.2026).

5. React. Documentation. URL: https://react.dev/ (дата обращения: 12.09.2026).